



Support de cours du module théorie des langages

Opérations sur les AEF

Présenté par Dr. T. Benouhiba

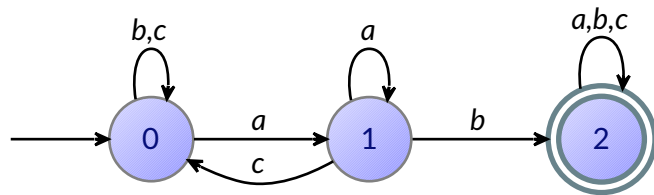




Complément

- Problème : étant donné l'AEF acceptant un langage L , comment construire l'AEF acceptant $\bar{L}$?

- On considère un exemple d'AEF :
 - Quel est le langage accepté ?

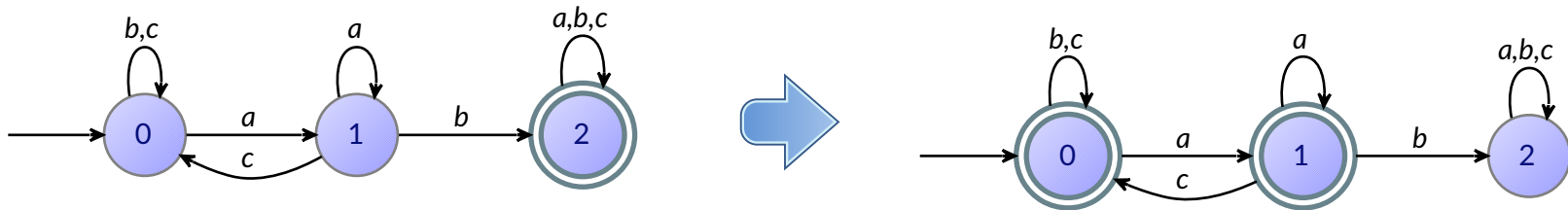


- Analysons le mot **cabc** :
 - $(0,c) \rightarrow (0,a) \rightarrow (1,b) \rightarrow (2,c) \rightarrow (2,\epsilon)$  L'état 2 est final
- Analysons le mot **acb** :
 - $(0,a) \rightarrow (1,c) \rightarrow (0,b) \rightarrow (0,\epsilon)$  L'état 0 n'est pas final



Complément

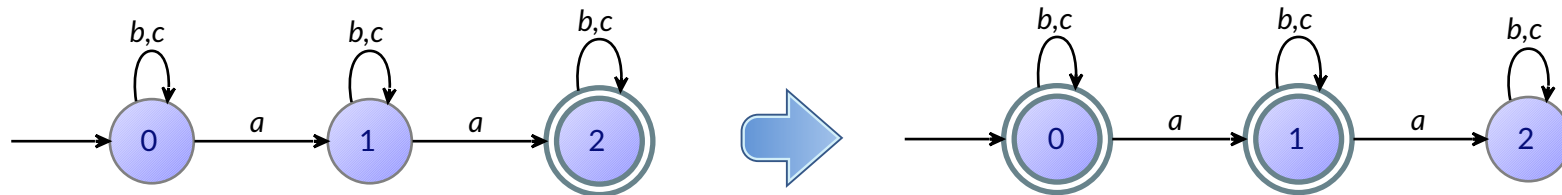
- L'automate permet alors de distinguer entre L et $\bar{L}$ grâce aux états finaux
- Solution : basculer le statut final des états
 - États non-finaux deviennent finaux
 - États finaux deviennent non-finaux
- Application (est-ce que l'AEF accepte vraiment le complément ?)





Complément

- Application sur un autre exemple :
 - Quel est le langage accepté ?

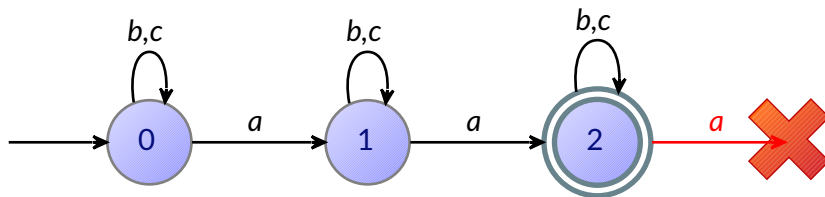


- Est-ce qu'on obtient l'AEF du complément ?
 - C'est quoi le problème ?
 - L'automate n'était pas **complet**



Complément

- Définition : un AEF est dit complet si la fonction de transition est définie pour tout état et pour tout symbole de son alphabet (en d'autres termes, il n'y a pas de transitions qui manquent)
- Dans cet automate, la transition $(2,a)$ n'est pas définie, l'AEF est donc incomplet

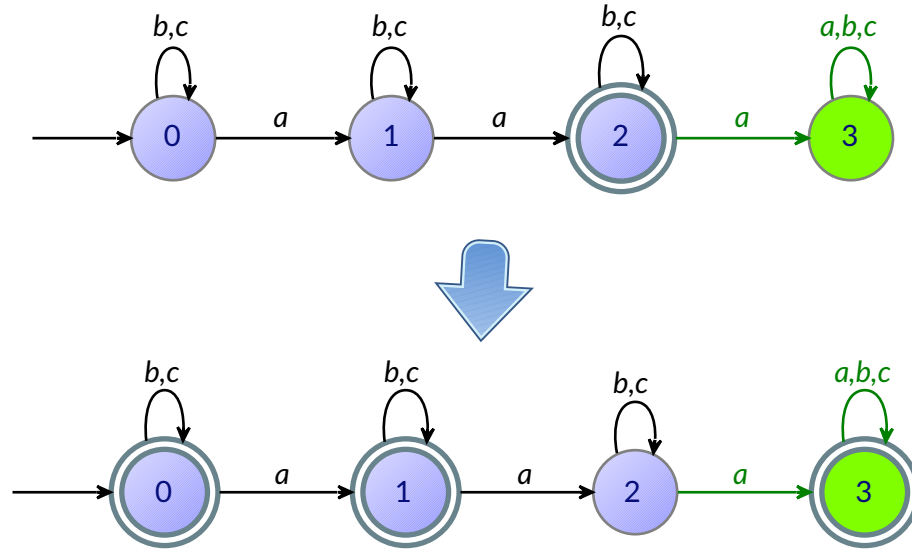


- Solution : rendre l'AEF complet
 - Rajouter un nouvel état (quelques soient le nombre de transitions manquantes)
 - Toutes les transitions manquantes mènent au nouvel état
 - Toutes les transitions sortantes du nouvel état reviennent au même état



Complément

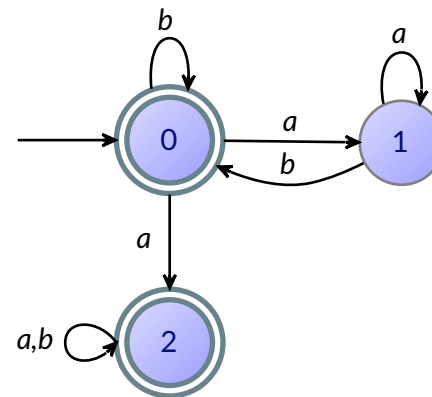
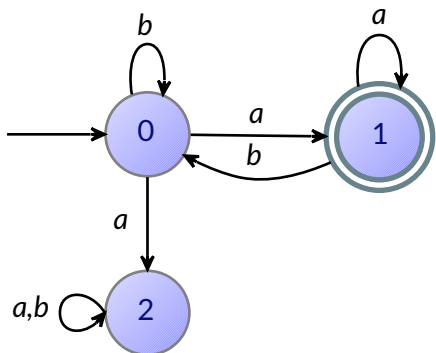
- On rend complet l'AEF puis on déduit l'AEF du langage complémentaire (on fait les vérifications...)





Complément

- Un troisième exemple :
 - Quel est l'automate qui accepte le mot **a** ?
 - Est-ce vraiment l'AEF du langage complémentaire ?
 - Le problème est le non-déterminisme !
 - Solution : rendre l'AEF déterministe





Complément : récapitulatif

- Pour obtenir l'AEF du langage complémentaire
 - Les états finaux deviennent non-finaux
 - Les états non-finaux deviennent finaux
- Conditions :
 - L'AEF doit être déterministe
 - L'AEF doit être complet



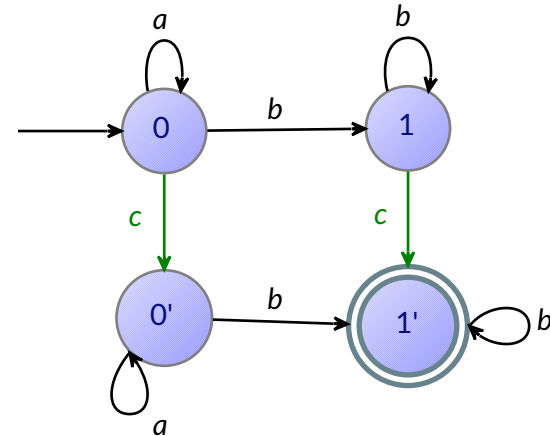
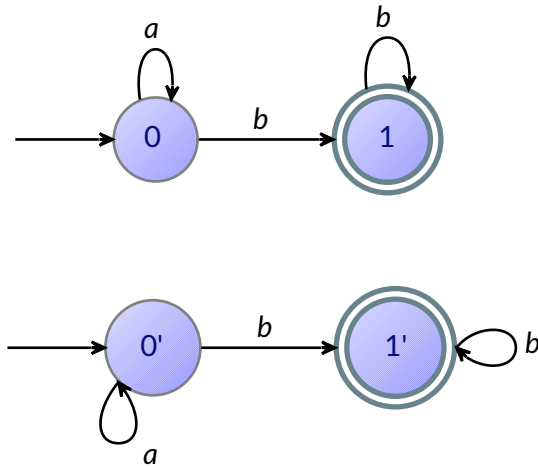
L'entrelacement

- Problème : on considère un langage L et un symbole a
 - Rappel : dans ce cas particulier, l'entrelacement de L et a ($L \sqcup a$) s'obtient en insérant le symbole a dans tout mot de L
- L'AEF de $L \sqcup a$ s'obtient à partir de celui de L (que l'on appelle ici A)
 - Créer une copie de l'AEF que l'on appelle A'
 - Primer tout état de A' (i devient i')
 - Les états finaux de A ne sont plus finaux
 - L'état initial de A' n'est plus initial
 - Relier tout état i de A avec son correspondant i' avec une transition étiquetée par a



L'entrelacement

- Application :
 - Il n'y a pas de pré-condition pour cette opération

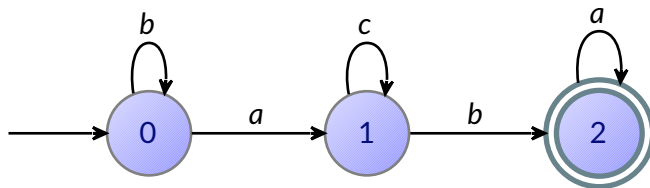




Automate du langage miroir

- Problème : étant donné l'AEF d'un langage L , comment construire l'AEF du langage miroir L^R ?

- Exemple :

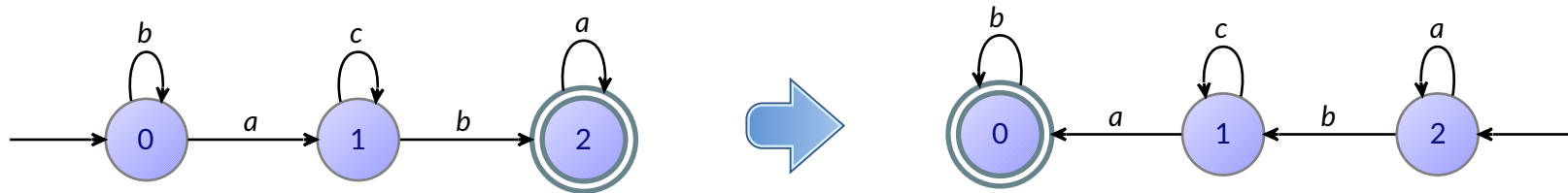


- Le langage accepté est : $\{b^p a^q b^r \mid p, q, r \geq 0\}$ (le langage miroir est : $\{a^r b^q a^p \mid p, q, r \geq 0\}$)
- Un mot est accepté par l'AEF s'il y a chemin commençant par l'état initial et se terminant par un état final
 - Si on parcourt ce chemin dans le sens inverse, on obtient le miroir du mot accepté



Automate du langage miroir

- Transformation :
 - L'état initial (états initiaux) devient (deviennent) final (finaux)
 - Les états finaux deviennent initiaux
 - On inverse les arcs, c'est-à-dire, que s'il y a un arc allant de l'état **i** vers **j** avec le symbole **a**, alors il sera redéfini pour aller de l'état **j** vers l'état **i** avec le même symbole **a**
 - L'existence de plusieurs états initiaux signifie que l'AEF est non-déterministe





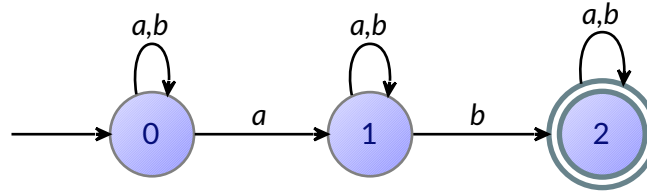
Application intéressante

- La transformation miroir est très simple tant en principe qu'en application, mais possède une application forte intéressante
 - Elle offre un deuxième algorithme de minimisation de n'importe quel AEF
- Algorithme de minimisation :
 - Soit un AEF A acceptant un langage L
 - Construire l'AEF de L^R , appelons cet AEF A_1
 - Déterminer A_1 , appelons l'AEF résultat A_2
 - Construire l'AEF du miroir à partir de A_2 , soit A_3 l'AEF résultant
 - Déterminer A_3 , le résultat représente l'AEF déterministe et minimal acceptant le langage L

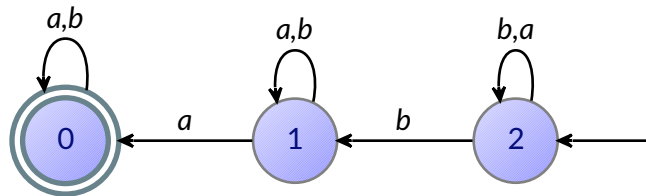


Exemple de minimisation

- On considère l'AEF suivant :
 - Quel est le langage accepté ?



- Minimisation :
 - 1) Construction miroir

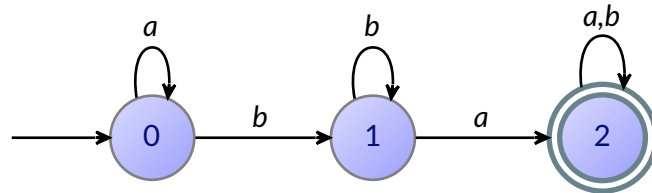




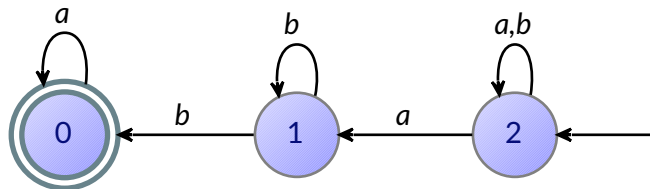
Exemple de minimisation

2) Déterminisation

Etat	a	b
2	2	1,2
1,2	0,1,2	1,2
0,1,2	0,1,2	0,1,2



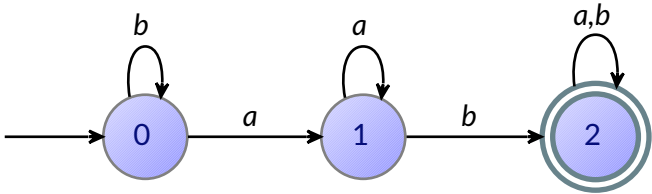
3) Construction miroir (on laisse tomber les primes pour simplifier) :



Exemple de minimisation

4) Détermination :

État	a	b
2	1,2	2
1,2	1,2	0,1,2
0,1,2	0,1,2	0,1,2





Produit d'automates

- On considère deux automates $A = (X, Q, q_0, F, \delta)$ et $A' = (X', Q', q_0', F', \delta')$ deux automates à états finis
- On appelle produit des deux automates A et A' l'automate $A'' = (X'', Q'', q_0'', F'', \delta'')$ défini comme suit :
 - $X'' = X \cap X'$
 - $Q'' = Q \times Q'$
 - $q_0'' = (q_0, q_0')$
 - $F'' = F \times F'$
 - $\delta''((q, q'), a) = \delta(q, a) \times \delta'(q', a)$



Produit d'automates

- Exemple :

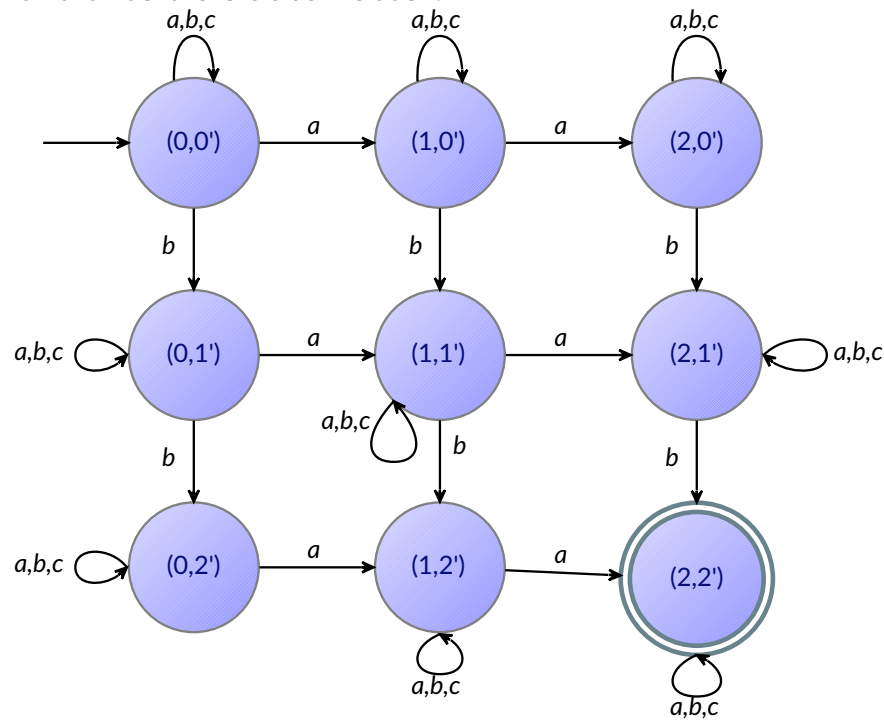


- L'état initial : $(0, 0')$
- Les états finaux : $\{2\} \times \{2'\} = \{(2, 2')\}$
- Exemple de calcul de la fonction de transition :
 - $\delta''((0, 0'), a) = \delta(0, a) \times \delta'(0', a) = \{0, 1\} \times \{0'\} = \{(0, 0'), (1, 0')\}$
 - $\delta''((0, 0'), c) = \delta(0, c) \times \delta'(0', c) = \{0\} \times \{0'\} = \{(0, 0')\}$



Produit d'automates

- On obtient l'AEF suivant :
 - Quel est la relation entre les trois automates ?





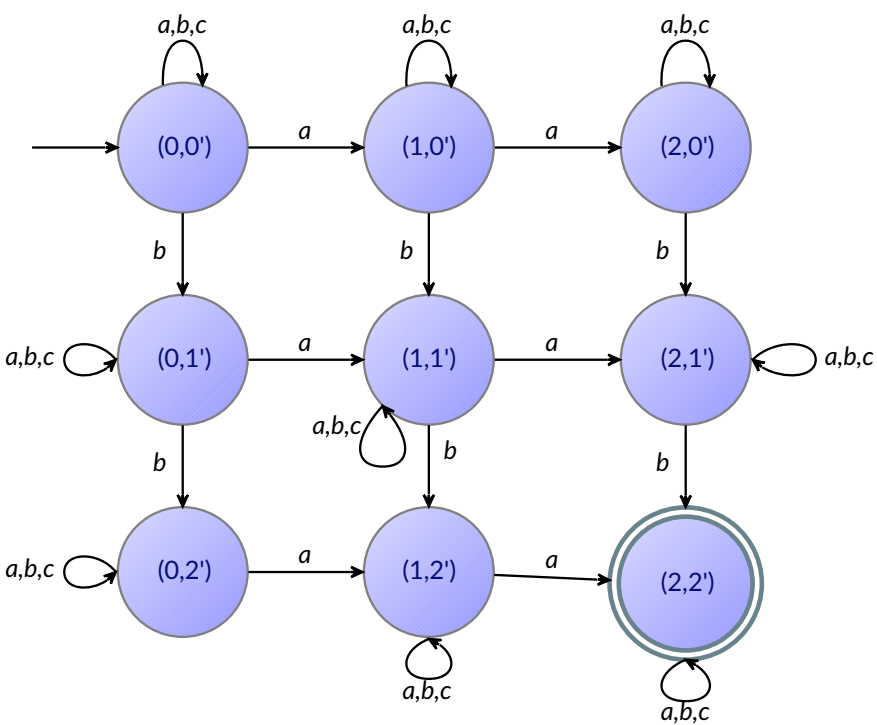
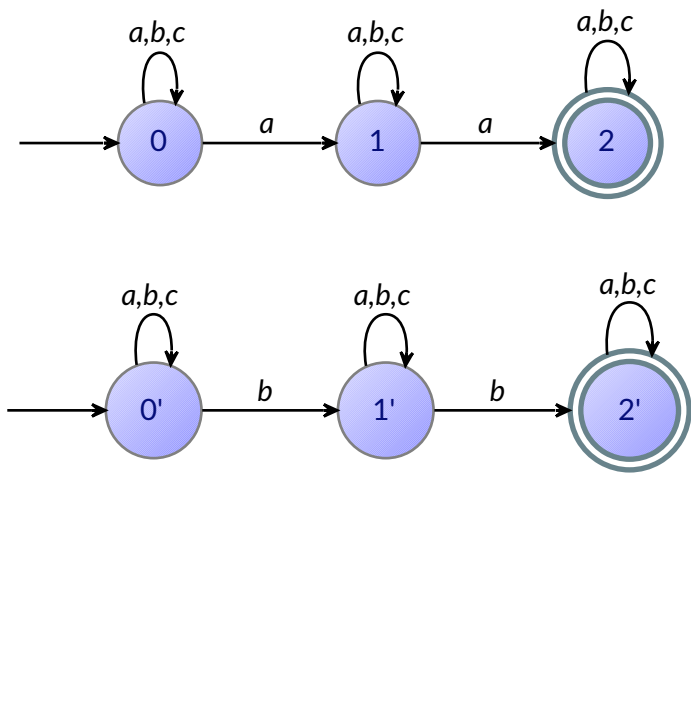
Propriété du produit

- Soit $w = w_1 w_2 \dots$ un mot accepté par l'automate produit :
 - On a : $((0, 0'), w_1) \rightarrow \dots ((q_f, q'_f), \epsilon)$ tel que q_f est un état final du premier automate et q'_f est un état final du deuxième automate
 - On a alors :
 - $(0, w_1) \rightarrow \dots (q_f, \epsilon)$ pour le premier automate (donc il est accepté par le premier automate)
 - $(0', w_1) \rightarrow \dots (q'_f, \epsilon)$ pour le deuxième automate (donc il est accepté par le deuxième automate)
 - Alors, l'automate produit accepte un mot si et seulement si les deux automates l'accepte
 - Il accepte alors l'intersection des langages acceptés par les deux automates



Produit des automates

- Vérification avec l'exemple :





Simulation d'un AEF

- On veut construire un programme, en langage Python, simulant un AEF déterministe
 - Il accepte en entrée une chaîne de caractères
 - En sortie, il affiche « accepté » ou « rejeté » selon que l'AEF accepte ou rejette le mot



Simulation d'un AEF

- Simulation des parties de la machine :
 - Ruban $\Rightarrow$ une chaîne de caractères que l'on appellera word
 - Déclaration (on la suppose globale) : `word=...`
 - La tête de lecture $\Rightarrow$ un entier `head` indiquant l'indice de la case actuelle du ruban (on la suppose globale)
 - Initialisation : `head=0`



Simulation d'un AEF

- Simulation des parties de la machine :
 - Déplacement de la tête : une fonction `next(...)` renvoyant le caractère en cours de lecture si non fin du ruban, sinon `\0`.

```
def next()
    if head < len(word) :
        c = word[head]
        head += 1
        return c
    else : return '\0'
```



Simulation d'un AEF

Soit à simuler un AEF :

$A=(X, Q, q_0, F, \delta)$, . Le code qui le simule aura la forme $\Rightarrow$

```
state=q0
c=next()
go=True
while go and c!='\0' :
    #Pour chaque état q
    mettre : if/elif state==q :
        #Pour chaque symbole a tel que  $\delta(q, a) = q'$ 
        mettre : if/elif c==a: state=q'
        mettre : else : go=False
    if go : c=next()
if c=='\0' and state in F : print("Accepted")
else : print("Rejected")
```



Simulation d'un AEF

- On va simuler l'AEF suivant :

